Practical:02	Implementation and Time analysis of linear and binary search algorithm.
__/__ /2026	

Objective:

To implement linear and binary search algorithms and evaluate their practical execution time and theoretical complexities relative to input size.

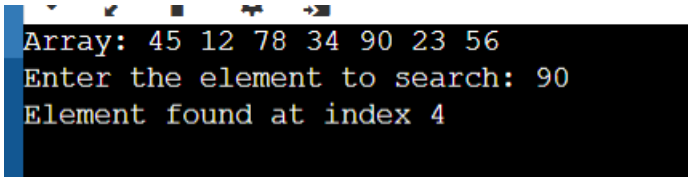
Program:

a) Linear Search

```
#include <iostream>
using namespace std;
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key) {
            return i;   // return index if found
        }
    }
    return -1;          // not found
}
int main() {
    int arr[] = {45, 12, 78, 34, 90, 23, 56};
    int n = sizeof(arr) / sizeof(arr[0]);
    int key;
    cout << "Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << "\nEnter the element to search: ";
    cin >> key;
    int result = linearSearch(arr, n, key);
    if (result != -1)
        cout << "Element found at index " << result << endl;
    else
        cout << "Element not found" << endl;
```

```
    return 0;
}
```

Output:

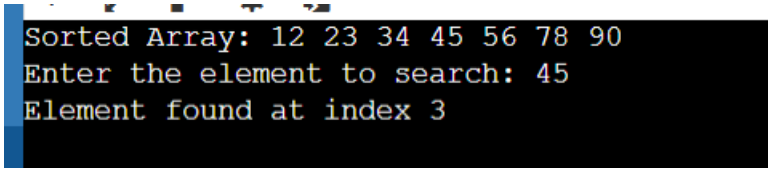


b) Binary Search

```
#include <iostream>
using namespace std;
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;  // avoids overflow
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;   // not found
}
int main() {
    int arr[] = {12, 23, 34, 45, 56, 78, 90};  // must be sorted
    int n = sizeof(arr) / sizeof(arr[0]);
    int key;
    cout << "Sorted Array: ";
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
    cout << "\nEnter the element to search: ";
    cin >> key;
```

```
    int result = binarySearch(arr, n, key);
    if (result != -1)
        cout << "Element found at index " << result << endl;
    else
        cout << "Element not found" << endl;
    return 0;
}
```

Output:



Analysis:

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$

Conclusion:

Practical testing shows that linear search is simple and works on any list, but becomes very slow as the dataset grows. For large datasets, binary search is much faster and more efficient because it cuts the search space in half, though it requires the data to be sorted first.